

Help C

❖ Tipos de Dados:

Char ----->	Caractere	[0 a 255]
Byte ----->	Inteiro para valores muito curtos	[-127 a 127]
Short ----->	Inteiro para valores curtos	[-32768 a 32767]
Int ----->	Números Inteiros	[até 2,000,000]
Long ----->	Inteiro para valores altos	[até 10 ¹¹]
Float ----->	Números Reais	[até 10 ³³]
Double ----->	Reais para valores altos	[até 10 ³³³]

❖ Especificadores de Formato:

%c Caractere	%u Decimal Sem Sinal	%d Inteiro Decimal	%f Real	%s String	%p Endereço
%o Inteiro Octal	%ld Inteiro Longo	%x Inteiro Hexadecimal	%lf Real Longo	%% Sinal de %	

❖ Exibição e Formatação:

Scanf("%d %c", &Variável);

Ler as informações digitadas. Onde primeiro deveremos utilizar o especificador de formato, em seguida a variável. Utilizar "&" nas variáveis do scanf.

fgets(variável, sizeof(variável), stdin);

variável[**strncpy**(variável, "\n")] = '\0';

Ler strings digitadas pelo usuário, sendo necessário passar a variável, sizeof da variável e stdin. Recomendado utilizar a função para remover o \n automático logo após a linha do fgets.

OBS: para textos é recomendado utilizar o fgets ao invés do scanf.

Printf ("Texto %d", Variável);

Exibe as informações no terminal. Onde dentro do " " deverá ser colocado o texto. Caso precise exibir a informação de uma variável, utilizar o especificador de formato e fora da " " seguindo a ordem, deverá ser colocada a variável.

OBS: caso queira exibir o endereço da variável, colocar & antes da variável no printf.

Formatar números reais: O valor das casas decimais deverá ser colocado entre o % e o f. ex: %0.2f, para duas casa decimais após a virgula.

\b O Cursor retrocede uma coluna	\t O Cursor avança para próxima linha
\r O Cursor Retrocede Para a primeira coluna da linha	\\ Exibe uma única barra invertida
\n O Cursor avança para uma nova linha	\" Exibe uma única aspa
\' Exibe um apóstrofo	\0 Para não exibir lixo de memória.

Cores no terminal:

Printf("\\033[1:31m");

Linhas	Fundo	Definição
[30] - Preto	[40] - Preto	Utilizado para colocar cores no terminal. Sendo o primeiro numero 1 = Negrito e 4 = Sublinhado. PS: deve ser utilizado apenas dentro do printf. Para reverter "\\033[0m".
[31] - Vermelho	[41] - Vermelho	
[32] - Verde	[42] - Verde	
[33] - Amarelo	[43] - Amarelo	
[34] - Azul	[44] - Azul	
[35] - Magenta	[45] - Magenta	
[36] - Ciano	[46] - Ciano	
[37] - Branco	[37] - Branco	

PS: para usar o printf, scanf, fgets é necessário importar a biblioteca: #include <stdio.h>

❖ Operadores Aritméticos:

+ Soma	= Atribuir	&& Conjunção (e / and)
- Subtração	== Comparar	Disjunção (ou / or)
/ Divisão	!= Diferente	+= Você Recebe
* Multiplicação	% Resto	! Negação
-- Decremento	<= Menor ou Igual	
++ Incremento	>= Maior ou Igual	

❖ Condição:

If - Else	Switch case
EX: <pre>int x = 10; if (x > 0) { printf("Número positivo"); } else { printf("Número negativo ou zero"); }</pre> OBS: você poderá substituir o: <pre>} else { } else if (condição){ if (condição){ ----> }</pre>	EX: <pre>int dia = 2; switch (dia) { case 1: printf("Sábado"); break; case 2: printf("Domingo"); break; default: printf("Outro dia"); }</pre> Definição: O switch permite escolher entre várias opções com base no valor de uma variável. Cada case representa uma possibilidade, e o default é usado se nenhuma for atendida.
Definição: O if-else é usado para tomar decisões com base em condições. Se a condição for verdadeira, um bloco de código é executado; caso contrário, o bloco do else é executado.	

❖ Repetição:

While	Do - While	For
EX: <pre>int i = 0; while (i < 3) { printf("%d\n", i); i++; }</pre> Definição: O while executa um bloco de código repetidamente enquanto a condição for verdadeira. A verificação acontece antes da execução.	EX: <pre>int i = 0; do { printf("%d\n", i); i++; } while (i < 3);</pre> Definição: O do-while também repete um bloco de código, mas a condição é verificada após a execução. Por isso, o código é executado pelo menos uma vez.	EX: <pre>int i; for (i = 0; i < 3; i++) { printf("%d\n", i); }</pre> Definição: O for é usado para loops com um número conhecido de repetições. Ele possui três partes: inicialização, condição e incremento.

❖ Vetor e Matriz:

Vetor	Matriz
Um vetor é uma lista de elementos de um único tipo, organizados em uma linha (estrutura unidimensional). Cada posição do vetor é chamada de índice, e os índices começam sempre do 0. Declaração: int vetor[5]; -----> Vetor com 5 espaços. EX: #include <stdio.h> int main() { int i, vet[5] = {2, 4, 6, 8, 10}; for (i = 0; i < 5; i++) { printf("Elemento %d: %d\n", i, vet[i]); } return 0; }	Uma matriz é como uma tabela de linhas e colunas, ou seja, uma estrutura bidimensional. É usada quando precisamos organizar dados em duas dimensões (ex: notas de alunos, tabuleiros, imagens etc.). Declaração: Int mat[5][5]; -----> Matriz com 5 linhas e 5 colunas. EX: #include <stdio.h> int main() { int i, j; int mat[2][3] = {{1, 2, 3}, {4, 5, 6}}; for (i = 0; i < 2; i++) { for (j = 0; j < 3; j++) { printf("%d ", mat[i][j]); } printf("\n"); } return 0; }

PS: (não obrigatório!) usar "i" como variável do vetor e "i" e "j" para matriz.

❖ Struct:

Organiza tipos primitivos como uma função, para caber mais variáveis, similar a um ficheiro. Struct aluno { Int mat; Char nome[30] }; Typedef -----> Renomeia funções. Ex: typedef struct aluno Aluno; EX: Aluno Turma[5]; Turma[0].mat	OBS: Sintaxe mais compactada entre o struct e typedef. Typedef struct { Int mat; Char nome[30]; } Aluno;
---	---

❖ Ponteiros:

Definição Ponteiros são identificados por * logo após a variável. Ponteiros em C são variáveis que guardam endereços de memória. Eles permitem acessar e modificar valores diretamente na memória.	EX: int x = 10; ---> posição(1000) int *p = &x; --> ou seja, a posição onde x se encontra. printf("%d", p); = 1000
---	---

❖ Alocação Dinâmica:

Funções:

Sizeof -----> Retorna o valor em bytes do tipo. EX: int = 4
Free(var) -----> liberar a memória, depois da execução.
Malloc -----> Aloca bytes de forma dinâmica. EX: (tipo*) malloc (variável * sizeof (tipo));
Calloc -----> Aloca bytes de forma dinâmica. Já inicia o vetor zerado. EX: (tipo*) calloc (variável, sizeof (tipo));
Realloc -----> Aloca bytes de forma dinâmica. Aloca durante execução. EX: (tipo*) realloc (variável, variável * sizeof (tipo));

❖ Modularização:

Função com Tipos	Função Void
Para declarar uma função devemos colocar o tipo antes do identificador da função, em seguida colocar "()". O tipo refere-se ao tipo do valor retornado pela função. Os parênteses são os parâmetros da função, parâmetros são os valores que a função recebe. Ex: <pre>int var (int n){ int i, res=1; for (i=1; i<=n; i++){ res = i * res; } return res; }</pre>	Void são funções que não retornam valores. Ex: <pre>Void menu(){ Printf("====Menu===="); Printf("Soma [1]"); Printf("Multiplicação [2]"); Printf("Divisão [3]"); Printf("Subtração [4]"); Printf("===="); }</pre>
Funções Recursivas	
Funções Recursivas fazem chamadas a se mesmo, servindo para otimizar um código. Ex: <pre>Int fatorial (int n) { If (n == 0){ return 1; } else { return n * fatorial (n - 1); } }</pre>	

❖ Passagem por valor e referência:

Definição:	
A linguagem de programação C permite que os parâmetros sejam passados para as funções de duas maneiras, por valor e por referência. Na passagem por valor uma expressão pode ser utilizada na chamada. Na passagem por referência, o endereço de uma variável deve ser passado na chamada da função. Dessa forma, a função pode modificar a variável diretamente.	
Passagem por valor: É a forma mais comum utilizada para passagem de parâmetros. Na passagem por valor uma expressão pode ser utilizada na chamada. EX: <pre>#include <stdio.h> #include <stdlib.h> Int soma(int x, int y){ return (x+y); } Int main(){ Int num1=2, num2=3; res; res = soma(num1, num2); printf("Resultado = %d", res); }</pre>	Passagem por referência: Sempre que possível é recomendável utilizar a forma de passagem por valor, para evitar "efeitos colaterais". EX: <pre>#include <stdio.h> #include <stdlib.h> Int soma(int *a, int *b, int *r){ *r = *a + *b; } Int main(){ Int num1=2, num2=3; res=0; soma(&num1, &num2, &res); printf("Resultado = %d", res); }</pre>

❖ Parâmetros do Main:

Definição	
Parâmetros do main em C são usados para receber valores da linha de comando quando o programa é executado. Eles permitem que o usuário envie informações externas para o programa. A forma mais comum é: <code>int main(int argc, char *argv[])</code> argc -----> contador para a quantidade de valores passados. argv[] -----> valores digitados pelo usuário.	OBS: o <code>argv[0]</code> é dedicado ao nome do arquivo, e os demais <code>argv[1]</code> , <code>argv[2]</code> ,...etc. são os valores passados. Para executar o arquivo abra o cmd, coloque o comando "cd" em seguida o local do arquivo. Depois faça o executável. Use o comando "gcc" logo após o nome do arquivo .c, em seguida o comando "-o", e por fim o nome do arquivo .exe. Para executar você deverá passar o nome do arquivo e os demais parâmetros.
Código: <pre>#include <stdio.h> #include <stdlib.h> int main(int argc, char *argv[]) { if (argc < 2) { printf("Uso: %s num1 num2 ...\n", argv[0]); return 1; } int soma = 0; for (int i = 1; i < argc; i++) { soma += atoi(argv[i]); } printf("Soma = %d\n", soma); return 0; }</pre>	Execução pelo cmd: <pre>cd C:\Users\erric\Documents\Estudos em TI\C gcc main.c -o main.exe main 2 3 4</pre> Resultado: A soma de 2 + 3 + 4 = 9

❖ Bibliotecas:

Para utilizar as bibliotecas corretamente, você deverá utilizar o “#include” antes do nome da biblioteca.

EX:

#include <stdio.h>

<stdio.h>	<stdlib.h>
printf() Exibe dados formados na saída padrão. scanf() Lê dados formados da entrada padrão. fgets() Lê uma linha de texto da entrada padrão. fflush(stdin) Tenta remover o buffer das palavras.	malloc() Aloca dinamicamente um bloco de memória. calloc() Aloca dinamicamente um bloco de iniciando com 0. realloc() Redimensiona um bloco de memória. free() Libera a memória alocada dinamicamente.
<math.h>	<ctype.h>
pow(x, y) x elevado à potência de y. sqrt(x) Raiz quadrada de x. ceil() Arredonda para cima. floor() Arredonda para baixo.	toupper() Converte caracteres minúsculos em maiúsculos. tolower() Converte caracteres maiúsculos em minúsculos. isalpha(c) verifica se c é uma letra. isdigit(c) verifica se c é um dígito decimal.
<time.h>	isalnum(c) verifica se c é uma letra ou um número. isspace(c) verifica se c é um caractere de espaço em branco. islower(c) verifica se c é uma letra minúscula. isupper(c) verifica se c é uma letra maiúscula.
time() Obtém o tempo atual. difftime(end, start) retorna a diferença em segundos entre dois pontos. clock() Retorna o tempo de CPU usado desde o inicio do programa. (em ticks)	

❖ Arquivos:

Definição: Em linguagem C, arquivos são usados para armazenar dados de forma permanente, ou seja, os dados não desaparecem quando o programa é encerrado. Diferente das variáveis, que ficam na memória RAM, os arquivos são guardados no disco (HD ou SSD). Para trabalhar com arquivos, utilizamos um ponteiro do tipo FILE, definido na biblioteca <stdio.h>. EX: FILE *arquivo;		Definição: <table><tr><th>Mod o</th><th>Função</th><th>Descrição</th></tr><tr><td>"w"</td><td>Criar</td><td>Cria ou sobrescreve arquivo</td></tr><tr><td>"a"</td><td>Adicionar</td><td>Abre para acrescentar dados</td></tr><tr><td>"r"</td><td>Leitura</td><td>Abre para leitura.</td></tr></table> Após usar o arquivo, é obrigatório fechá-lo com fclose() para liberar os recursos e garantir que os dados sejam gravados corretamente. EX: fclose(arquivo);		Mod o	Função	Descrição	"w"	Criar	Cria ou sobrescreve arquivo	"a"	Adicionar	Abre para acrescentar dados	"r"	Leitura	Abre para leitura.
Mod o	Função	Descrição													
"w"	Criar	Cria ou sobrescreve arquivo													
"a"	Adicionar	Abre para acrescentar dados													
"r"	Leitura	Abre para leitura.													
Exemplos: <pre>#include <stdio.h> #include <stdlib.h> int main() { FILE *arquivo; char texto[100]; arquivo = fopen("exemplo.txt", "w"); if (arquivo == NULL) { printf("Erro ao criar o arquivo!\n"); return 1; } fprintf(arquivo, "Olá, este é um exemplo de arquivo em C!\n"); fprintf(arquivo, "Gravando dados com sucesso.\n"); fclose(arquivo); printf("Dados gravados no arquivo.\n\n"); arquivo = fopen("exemplo.txt", "r"); if (arquivo == NULL) { printf("Erro ao abrir o arquivo para leitura!\n"); return 1; } printf("Conteúdo do arquivo:\n"); while (fgets(texto, 100, arquivo) != NULL) { printf("%s", texto); } fclose(arquivo); return 0; }</pre>	Funções: fopen() Abrir arquivo. fclose() Fechar arquivo. fputc() Escrever um caracter fputs() Escrever uma string fgetc() Lê um caracter fseek() Posicionar em um byte específico fprintf() Escreve uma cadeia		Funções: fscanf() Lê uma cadeia fgets() Lê uma frase feof() End Of File ? (Fim de Arquivo ?) ferror() Ocorreu um erro ? rewind() Reloca o indicador para o início remove() Apaga o arquivo fflush() Descarrega o arquivo												

❖ TAD (Projeto):

Definição: Um TAD (Tipo Abstrato de Dados) é uma forma de organizar e dividir o código em partes independentes, separando a lógica interna dos dados da forma como eles são usados. Na prática, em C, isso é feito dividindo o projeto em três arquivos principais: Arquivo de Cabeçalho (.h) → Define a interface (o que o módulo oferece). Arquivo de Implementação (.c) → Contém o código das funções (como elas funcionam). Arquivo Principal (main.c) → Usa o TAD (chama as funções definidas).	Como utilizar: [main]: é a parte do usuário, é onde será feita a chamada das funções. [implementação]: é onde estão as funções, basicamente toda a lógica do seu código estará aqui. [cabeçalho]: é uma espécie de ponte, que faz a conexão entre o main e a implementação. O cabeçalho terá que chamar as funções e também terá o struct. OBS: no main e implementação, terá que ser chamado um include do cabeçalho. #include "pls.h"
--	--

❖ Diretivas:

Diretivas normais definição:

Diretivas são instruções especiais que orientam o pré-processador antes da compilação do programa em C. Elas não fazem parte do código executável, ou seja, não são comandos C comuns, mas instruções que dizem ao compilador como preparar o código antes de traduzi-lo para linguagem de máquina.

EX:

Incluir bibliotecas no programa (`#include`);

Definir constantes (`#define`);

Criar macros, que são pequenos trechos de código substituídos automaticamente antes da compilação;

Controlar a compilação condicional, permitindo compilar partes específicas do código com `#if`, `#ifdef`, `#endif`, etc.

Como usar:

Toda diretiva começa com o símbolo `#` (cerquilha) e não termina com ponto e vírgula. Elas são processadas antes da execução do programa pelo pré-processador, que substitui, insere ou remove partes do código conforme as diretivas indicam.

Diretivas normais exemplo:

```
#include <stdio.h> // Diretiva para incluir a
biblioteca padrão de entrada e saída
#define PI 3.14159 // Diretiva para definir uma
constante
```

```
int main() {
    float raio = 5.0;
    float area = PI * raio * raio;

    printf("A área do círculo é: %.2f\n", area);
    return 0;
}
```

Diretivas condicionais definição:

As diretivas condicionais permitem controlar quais partes do código serão compiladas, dependendo de certas condições. Isso é útil quando o mesmo programa precisa ser compilado em diferentes sistemas operacionais, versões ou configurações (por exemplo, modo de teste e modo final).

`#ifdef`

Significa "se definido". Serve para verificar se um identificador (como uma constante ou macro) foi declarado com `#define`. Se ele existir, o bloco de código logo abaixo será compilado.

`#ifndef`

Significa "se não definido". É o oposto de `#ifdef`. O código dentro do bloco só será compilado se o identificador não tiver sido definido.

`#if`

Permite testar expressões lógicas ou numéricas. Se a condição for verdadeira (diferente de zero), o código dentro do bloco será compilado.

`#else`

Funciona como o "senão" de um `if` comum. É usado junto com `#if`, `#ifdef` ou `#ifndef` para tratar o caso contrário.

`#elif`

É uma forma abreviada de "else if" para diretivas. Permite testar outras condições, caso a primeira `#if` não seja verdadeira.

`#endif`

Indica o fim de um bloco condicional iniciado por `#if`, `#ifdef` ou `#ifndef`. Sem ele, o compilador não saberá onde termina a condição.

Diretivas condicionais exemplo:

```
#include <stdio.h>

#define MODO_TESTE // Definindo uma
constante para simular o modo de teste

int main() {

#ifdef MODO_TESTE
    printf("Programa em modo de teste.\n");
#else
    printf("Programa em modo de produção.\n");
#endif

    return 0;
}
```

❖ Métodos de Busca:

Sequencial:	Binário:
A busca sequencial é o método mais simples de procurar um valor em um vetor. O algoritmo percorre todos os elementos, um por um, até encontrar o valor desejado ou chegar ao final do vetor. É simples, mas pouco eficiente para vetores grandes.	A busca binária é usada em vetores ordenados. Ela compara o valor buscado com o elemento central do vetor e decide procurar na metade esquerda ou direita, reduzindo o tempo de busca.
EX: <pre>#include <stdio.h> int buscaSequencial(int v[], int n, int x) { for(int i = 0; i < n; i++) if(v[i] == x) return i; return -1; } int main() { int v[] = {2, 4, 6, 8, 10}; int pos = buscaSequencial(v, 5, 6); printf("Posição: %d\n", pos); return 0; }</pre>	EX: <pre>#include <stdio.h> int buscaBinaria(int v[], int n, int x) { int ini = 0, fim = n - 1; while(ini <= fim) { int meio = (ini + fim) / 2; if(v[meio] == x) return meio; else if(v[meio] < x) ini = meio + 1; else fim = meio - 1; } return -1; } int main() { int v[] = {1, 3, 5, 7, 9}; int pos = buscaBinaria(v, 5, 7); printf("Posição: %d\n", pos); return 0; }</pre>
Pior caso: (N)	Pior caso: (Log N)

❖ Métodos de Ordenação:

Bubble sort:	Selection sort:
O Bubble Sort (ou ordenação por bolha) compara pares de elementos adjacentes e os troca de posição se estiverem fora de ordem. Esse processo se repete até o vetor estar ordenado.	O Selection Sort encontra o menor elemento do vetor e o coloca na primeira posição, depois repete o processo para as próximas posições. É simples, mas não muito eficiente.
EX: <pre>#include <stdio.h> void bubbleSort(int v[], int n) { for(int i = 0; i < n - 1; i++) for(int j = 0; j < n - i - 1; j++) if(v[j] > v[j + 1]) { int temp = v[j]; v[j] = v[j + 1]; v[j + 1] = temp; } } int main() { int v[] = {5, 2, 9, 1, 3}; bubbleSort(v, 5); for(int i = 0; i < 5; i++) printf("%d ", v[i]); return 0; }</pre>	EX: <pre>#include <stdio.h> void selectionSort(int v[], int n) { for(int i = 0; i < n - 1; i++) { int min = i; for(int j = i + 1; j < n; j++) if(v[j] < v[min]) min = j; int temp = v[i]; v[i] = v[min]; v[min] = temp; } } int main() { int v[] = {4, 2, 7, 1, 3}; selectionSort(v, 5); for(int i = 0; i < 5; i++) printf("%d ", v[i]); return 0; }</pre>
Pior caso: (N²)	Pior caso: (N²)

❖ Métodos de Ordenação:

Quick sort:

O Quick Sort também usa o método dividir e conquistar. Ele escolhe um elemento pivô e organiza os outros elementos menores à esquerda e os maiores à direita, repetindo o processo recursivamente.

EX:

```
#include <stdio.h>
void quickSort(int v[], int ini, int fim) {
    int i = ini, j = fim, pivo = v[(ini + fim) / 2];
    while(i <= j) {
        while(v[i] < pivo) i++;
        while(v[j] > pivo) j--;
        if(i <= j) {
            int temp = v[i];
            v[i] = v[j];
            v[j] = temp;
            i++; j--;
        }
    }
    if(ini < j) quickSort(v, ini, j);
    if(i < fim) quickSort(v, i, fim);
}
int main() {
    int v[] = {7, 2, 1, 6, 8, 5, 3, 4};
    quickSort(v, 0, 7);
    for(int i = 0; i < 8; i++) printf("%d ", v[i]);
    return 0;
}
```

Pior caso: (N^2)

Merge sort:

O Merge Sort usa o método dividir para conquistar: divide o vetor em partes menores, ordena cada uma e depois as combina em uma só lista ordenada. É eficiente, mesmo para grandes conjuntos.

EX:

```
#include <stdio.h>
void merge(int v[], int ini, int meio, int fim) {
    int i = ini, j = meio + 1, k = 0;
    int temp[fim - ini + 1];
    while(i <= meio && j <= fim)
        temp[k++] = (v[i] < v[j]) ? v[i++] : v[j++];
    while(i <= meio) temp[k++] = v[i++];
    while(j <= fim) temp[k++] = v[j++];
    for(i = ini, k = 0; i <= fim; i++, k++)
        v[i] = temp[k];
}
void mergeSort(int v[], int ini, int fim) {
    if(ini < fim) {
        int meio = (ini + fim) / 2;
        mergeSort(v, ini, meio);
        mergeSort(v, meio + 1, fim);
        merge(v, ini, meio, fim);
    }
}
int main() {
    int v[] = {8, 3, 5, 1, 7, 2};
    mergeSort(v, 0, 5);
    for(int i = 0; i < 6; i++) printf("%d ", v[i]);
    return 0;
}
```

Pior caso: ($N \log N$)

Insertion sort:

O Insertion Sort insere cada elemento na posição correta em relação aos anteriores, simulando a forma como organizamos cartas na mão. É eficiente para listas pequenas.

EX:

```
#include <stdio.h>
void insertionSort(int v[], int n) {
    for(int i = 1; i < n; i++) {
        int chave = v[i];
        int j = i - 1;
        while(j >= 0 && v[j] > chave) {
            v[j + 1] = v[j];
            j--;
        }
        v[j + 1] = chave;
    }
}
int main() {
    int v[] = {5, 3, 4, 1, 2};
    insertionSort(v, 5);
    for(int i = 0; i < 5; i++) printf("%d ", v[i]);
    return 0;
}
```

Pior caso: (N^2)

❖ Método Guloso: (lá ele)

Definição:

O método guloso (ou algoritmo guloso) é uma estratégia de resolução de problemas que toma sempre a melhor decisão local em cada etapa, com a esperança de que essa escolha leve à melhor solução global.

Em outras palavras, ele escolhe a opção mais vantajosa naquele momento, sem voltar atrás, buscando otimizar o resultado final de forma rápida e eficiente.

Um exemplo clássico é o problema do troco, onde o algoritmo guloso tenta dar o menor número de notas ou moedas escolhendo sempre o maior valor disponível que ainda cabe no troco restante.

EX:

```
#include <stdio.h>
int calcula(int troco, int qtdopcoes, int *notas){
    int total = 0;
    for(int i = qtdopcoes - 1; i >= 0; i--){ // qtd de opções
        int totalparc = troco / notas[i]; // qtd de notas por valor
        de nota
        troco -= totalparc * notas[i]; // troco atualizado a cada
        execução
        total += totalparc; // qtd de notas para o troco
        geral
        if(totalparc != 0){
            printf("%d nota*s de %d\n", totalparc, notas[i]);
        }
    }
    return total;
}

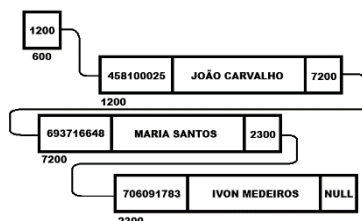
int main(){
    int qtdopcoes = 3, troco = 8, notas[3] = {1, 4, 5};
    printf("Qtd de notas: %d", calcula(troco, qtdopcoes, notas));
}
```

❖ Mapeamento:

Definição:

Em C, o termo mapeamento geralmente se refere à associação entre dois conjuntos de dados, onde cada elemento de um conjunto (a “chave”) é ligado a um elemento correspondente de outro conjunto (o “valor”).

Mapeamento é o processo de ligar uma chave única a um valor correspondente, permitindo procurar rapidamente informações associadas a essa chave.



EX:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    int chave;
    char valor[50];
} Par;

typedef struct {
    Par itens[100];
    int tamanho;
} Mapeamento;

// Função para criar um mapeamento
vazio
Mapeamento criar() {
    Mapeamento m;
    m.tamanho = 0;
    return m;
}

// Inserir um par (chave, valor)
void inserir(Mapeamento *m, int chave,
const char *valor) {
    m->itens[m->tamanho].chave = chave;
    strcpy(m->itens[m->tamanho].valor,
valor);
    m->tamanho++;
}
```

EX:

```
// Buscar um valor pela chave
char* buscar(Mapeamento *m, int
chave) {
    for (int i = 0; i < m->tamanho; i++) {
        if (m->itens[i].chave == chave) {
            return m->itens[i].valor;
        }
    }
    return NULL; // não encontrado
}

int main() {
    Mapeamento mapa = criar();

    inserir(&mapa, 1, "Maria");
    inserir(&mapa, 2, "João");
    inserir(&mapa, 3, "Ana");

    int id = 2;
    char *nome = buscar(&mapa, id);

    if (nome)
        printf("ID %d → Nome: %s\n", id,
nome);
    else
        printf("ID %d não encontrado.\n",
id);

    return 0;
}
```

❖ Estruturas de Dados [PILHAS]:

Pilhas:

Definição:

A pilha é uma estrutura de dados do tipo LIFO (Last In, First Out), ou seja, o último elemento a entrar é o primeiro a sair. Podemos imaginar uma pilha como uma pilha de pratos: o último prato colocado em cima é o primeiro a ser retirado.

Funções:

criarPilha()

Inicializa uma pilha, ou seja, prepara ela para começar a ser usada.

cheia()

Checa se a pilha está cheia, retorna 1 se a pilha estiver cheia e retorna 0 caso não esteja cheia.

vazia()

Checa se a pilha está vazia, retorna 1 se a pilha estiver vazia e retorna 0 caso não esteja vazia.

push()

Essa função coloca um novo elemento no topo da pilha. Antes de inserir, normalmente é preciso verificar se a pilha não está cheia (no caso de vetor).

pop()

Essa função remove o elemento que está no topo da pilha. Antes de remover, é preciso verificar se a pilha não está vazia.

top()

A função top (ou peek) retorna o valor que está no topo, sem remover o elemento da pilha.

mostrarPilha()

Exibe todas as informações que estiverem na pilha. PS: Não altera a pilha.

Código das funções da pilha:

```
void criarPilha(Pilha* p){
    p->topo = -1;
    printf("SUCESSO\n");
}

int cheia(Pilha p){
    return(p.topo == MAX - 1);
}

int vazia(Pilha p){
    return(p.topo == - 1);
}

void push(Pilha* p, int item){
    if(cheia(*p)){
        printf("OVERFLOW\n");
    }else{
        p->topo++;
        p->itens[p->topo] = item;
        printf("%d foi empilhado!\n",
item);
    }
}

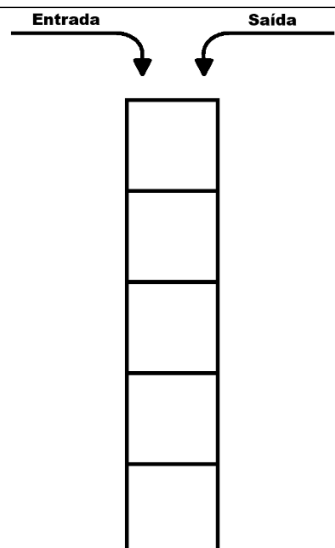
int pop(Pilha* p){
    int ret = -1;
    if(!vazia(*p)){
        ret = p->itens[p->topo];
        p->topo--;
    }
    return ret;
}

int top(Pilha p){
    if(!vazia(p)){
        return p.itens[p.topo];
    }
    return -1;
}

void mostrarPilha(Pilha p){
    printf("P:[ ");
    while(!vazia(p)){
        printf("%d ", pop(&p));
    }
    printf("]\n");
}
```

Conclusão:

Ela é usada em situações que exigem controle reverso, como nas chamadas de funções, operações de desfazer (undo) e processos recursivos. Em C, é implementada com funções como push() (empilha), pop() (desempilha) e top() (consulta o topo), permitindo armazenar e recuperar dados de forma organizada e eficiente.



❖ Estruturas de Dados [FILAS]:

Filas:

Definição:

A fila é uma estrutura de dados do tipo FIFO (First In, First Out), ou seja, o primeiro elemento a entrar é o primeiro a sair. Ela pode ser comparada a uma fila de pessoas: quem chega primeiro é atendido primeiro.

Funções:

criarPilha()

Inicializa uma fila, ou seja, prepara ela para começar a ser usada.

cheia()

Checa se a fila está cheia, retorna 1 se a pilha estiver cheia e retorna 0 caso não esteja cheia.

vazia()

Checa se a fila está vazia, retorna 1 se a pilha estiver vazia e retorna 0 caso não esteja vazia.

enqueue()

Ela é responsável por inserir um novo elemento no final da fila.

dequeue()

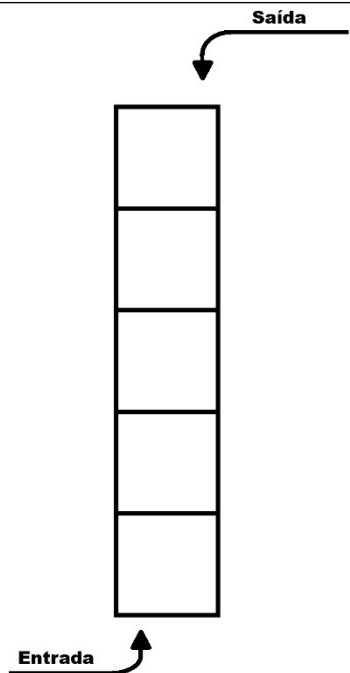
Remove o elemento do início, mantendo a ordem de chegada.

mostrar()

Exibe todas as informações que estiverem na fila. PS: Não altera a fila.

Conclusão:

Ela é utilizada em situações que exigem ordem de atendimento, como no gerenciamento de tarefas, processos de impressão, filas de atendimento, ou controle de requisições em sistemas. Em C, a fila é implementada com funções como enqueue() (inserir) e dequeue() (remover), garantindo que os dados sejam processados na mesma sequência em que chegam.



Código das funções da pilha:

```
void criarFila(Fila* f){
    f->ini = 0;
    f->fim = 0;
    f->total = 0;
    printf("Fila inicializada\n");
}

int cheia(Fila f){
    return(f.total == MAX);
}

int vazia(Fila f){
    return(f.total == 0);
}

void enqueue(Fila* f, int item){
    if(!cheia(*f)){
        f->dados[f->fim] = item;
        f->fim++;
        if(f->fim == MAX){
            f->fim = 0;
        }
        f->total++;
        printf("%d foi enfileirado\n", item);
    }else{
        printf("Fila cheia!\n");
    }
}

int dequeue(Fila* f){
    if(!vazia(*f)){
        int ret = f->dados[f->ini];
        f->ini++;
        if(f->ini == MAX){
            f->ini = 0;
        }
        f->total--;
        return ret;
    }
}

void mostrar(Fila f){
    printf("F:[ ");
    while(!vazia(f)){
        printf("%d ", dequeue(&f));
    }
    printf("]\n");
}
```

❖ Estruturas de Dados [Listas]:

Listas:

Definição:

Em C, listas são estruturas de dados dinâmicas usadas para armazenar e organizar elementos de forma encadeada. Diferente dos vetores, elas não têm tamanho fixo — os elementos (nós) são criados e ligados uns aos outros por ponteiros.

Cada nó geralmente contém dois campos. Dado (a informação que será armazenada). Ponteiro para o próximo nó (que indica onde está o próximo elemento da lista).

O primeiro elemento da lista é chamado de cabeça (head), e o último aponta para NULL, indicando o fim da lista.

Funções:

criarLista()

Cria uma lista vazia, fazendo o ponteiro inicial apontar para NULL.

vazia()

Verifica se a lista está vazia. Retorna 1 se estiver vazia, e 0 caso contrário.

inserirIni()

Insere um novo elemento no início da lista.

mostrar()

Percorre a lista e exibe todos os elementos na tela, um a um.

buscarL()

Procura um elemento específico na lista. Retorna o endereço do nó se encontrar o valor, ou NULL se não encontrar.

removerL()

Remove da lista o nó que contém o valor informado.

inserirOrdenado()

Insere o elemento em ordem crescente na lista.

Código das funções da pilha:

```
void criarLista(Lista* L){
    *L = NULL;
    printf("Sucesso!\n");
}

int vazia(Lista L){
    return (L == NULL);
}

void inserirIni(Lista* L, int item){
    Lista novo;
    novo = (Lista)malloc(sizeof(struct no));
    novo->info = item;
    novo->prox = *L;
    *L = novo;
    printf("%d foi inserido na lista\n", item);
}

void mostrar(Lista L){
    printf("L[ ");
    while(L != NULL){
        printf("%d ", L->info);
        L = L->prox;
    }
    printf("]\n");
}

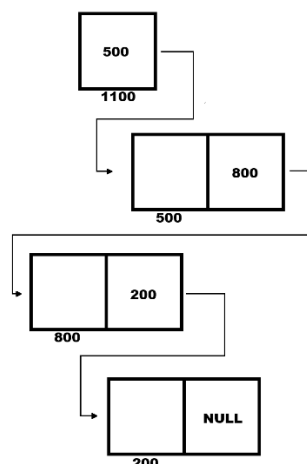
Lista buscarL(Lista L, int valor){
    while(L != NULL){
        if(L->info == valor)
            return L; // Encontrou o valor
        L = L->prox;
    }
    return NULL; // Não encontrado
}

void removerL(Lista* L, int valor){
    Lista atual = *L, anterior = NULL;
    while(atual != NULL && atual->info != valor){
        anterior = atual;
        atual = atual->prox;
    }
    if(atual == NULL){
        printf("Elemento %d não encontrado!\n", valor);
        return;
    }
    if(anterior == NULL){
        *L = atual->prox;
    }else{
        anterior->prox = atual->prox;
        free(atual);
        printf("Elemento %d removido com sucesso!\n", valor);
    }
}

void inserirOrdenado(Lista* L, int valor){
    Lista novo = (Lista)malloc(sizeof(struct no));
    novo->info = valor;
    novo->prox = NULL;
    Lista atual = *L, anterior = NULL;
    while(atual != NULL && atual->info < valor){
        anterior = atual;
        atual = atual->prox;
    }
    if(anterior == NULL){ // Inserir no início
        novo->prox = *L;
        *L = novo;
    } else {
        novo->prox = atual;
        anterior->prox = novo;
    }
    printf("%d inserido de forma ordenada!\n", valor);
}
```

Conclusão:

Na prática, as listas são muito utilizadas quando não se sabe a quantidade exata de dados que serão manipulados, como em sistemas de cadastro, filas de atendimento ou manipulação de dados variáveis em tempo de execução. O programador pode criar funções para inserir, remover, buscar e percorrer a lista, tornando o gerenciamento de memória mais eficiente que em vetores fixos.



❖ Estrutura de Dados [ÁRVORE]:

Árvore

Definição:

Em C, uma árvore é uma estrutura de dados hierárquica usada para armazenar informações de forma organizada, permitindo uma busca, inserção e remoção eficientes. Ela é composta por nós (nodes), onde cada nó pode conter um valor e ponteiros que indicam seus filhos.

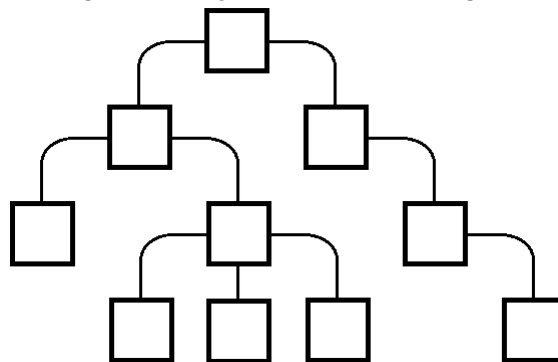
O nó principal é chamado de raiz (root), e os nós que não possuem filhos são chamados de folhas (leaves). Cada nó (exceto a raiz) tem exatamente um pai, formando uma estrutura semelhante a uma árvore invertida.

Em C, as árvores são implementadas utilizando ponteiros e estruturas (struct), já que a linguagem não possui esse tipo de dado nativamente.

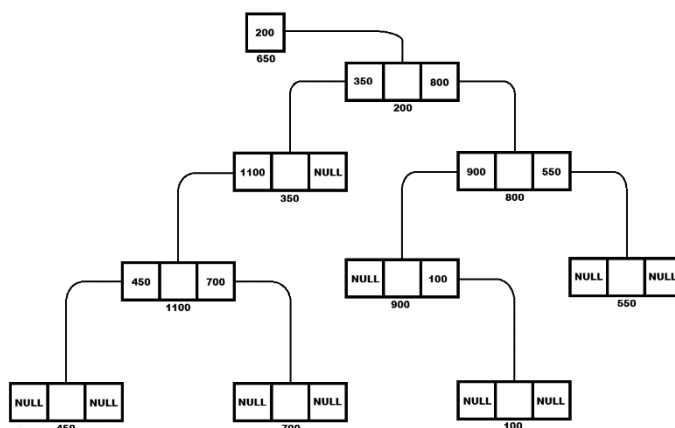
Com essa definição, é possível criar funções para inserir, buscar e percorrer os elementos da árvore. Por exemplo, em uma árvore binária de busca (BST), os valores menores que o nó atual ficam à esquerda, e os maiores ficam à direita.

Em resumo, as árvores em C são estruturas dinâmicas e flexíveis, amplamente usadas em sistemas de banco de dados, compiladores, jogos e algoritmos que exigem organização hierárquica ou busca rápida de informações.

Representação de forma simples:



Representação Com Ponteiros:



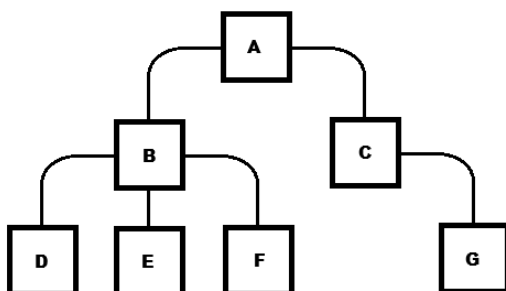
Fundamentos Básicos:

- GRAU - número de subárvores de um nó.
- FOLHA - um nó que possui grau zero, ou seja, não possui subárvores.
- FILHOS - são as raízes das subárvores de um nó.
- Nó não terminal - é um nó que não é uma folha e é diferente da raiz.

- A altura de uma árvore é o comprimento do caminho mais longo da raiz até uma das folhas.
- Uma árvore nula tem altura 0.
- Todos os nós são acessíveis a partir da raiz.
- Existe um único caminho entre a raiz e qualquer outro nó.

Formas de Representação:

Grafo (Representação Mais Utilizada)



Identação

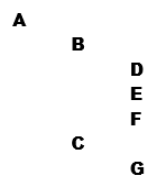
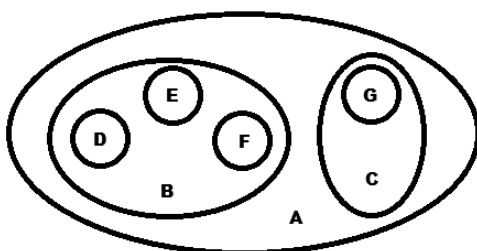


Diagrama de Venn (Ou Diagrama de Inclusão)



Parênteses Aninhados

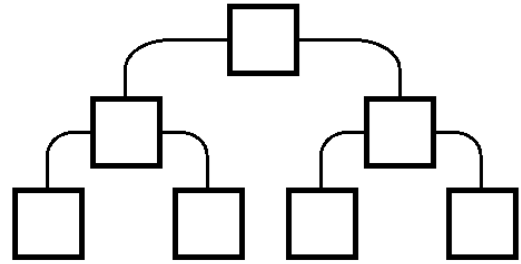
(A (B (D, E, F), C (G)))

Árvore Binária:

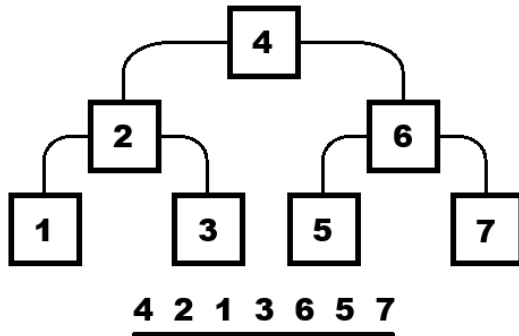
Definição:

Uma árvore binária é uma estrutura de dados hierárquica composta por nós, onde cada nó pode ter no máximo dois filhos: um à esquerda e outro à direita. Ela é usada para representar relações entre elementos de forma organizada, facilitando operações como percorrer, inserir e remover dados de maneira eficiente.

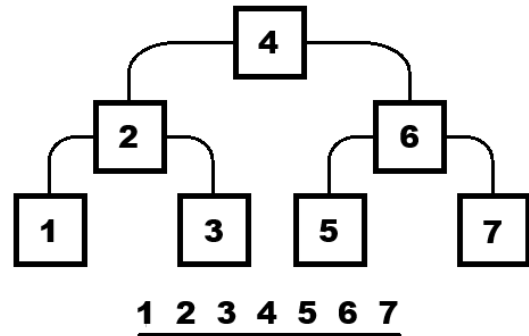
Representação:



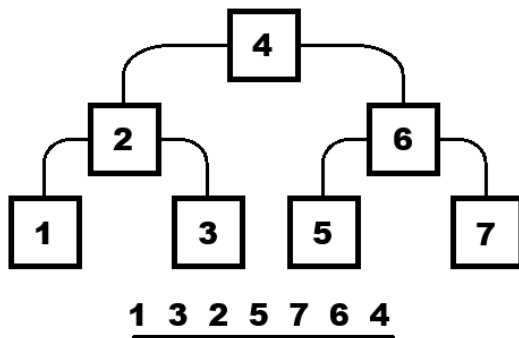
Prefixa



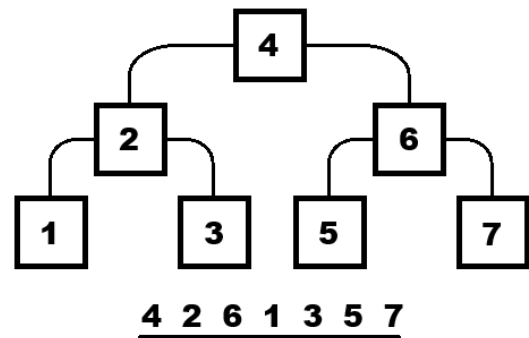
Infixa



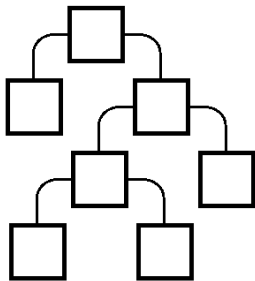
Posfixa



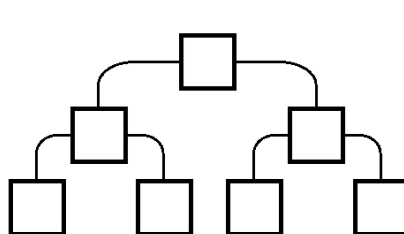
Em Nível



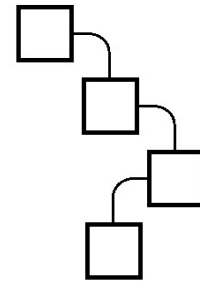
Estritamente Binária



Binária Cheia



Degenerada

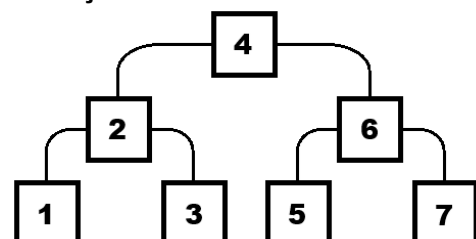


Árvore Binária de Busca (ABB)

Definição:

A árvore binária de busca é um tipo especial de árvore binária em que os elementos são armazenados de forma ordenada: todos os valores menores que o nó atual ficam à esquerda, e todos os valores maiores ficam à direita. Essa característica permite realizar buscas, inserções e remoções de forma rápida e eficiente.

Representação:



Funções:

Arv no(Arv A, Item x, Arv B)

Cria um novo nó da árvore. Recebe como parâmetros o filho esquerdo (A), o valor do item (x) e o filho direito (B). Retorna o ponteiro para o novo nó alocado dinamicamente.

void emOrdem(Arv A)

Percorre a árvore em ordem simétrica (in-order): visita primeiro o filho esquerdo, depois o nó atual e, por fim, o filho direito. Essa ordem exibe os elementos da árvore binária de busca em ordem crescente.

void preOrdem(Arv A)

Faz o percurso em pré-ordem (pre-order): exibe primeiro o nó atual, depois percorre o filho esquerdo e em seguida o direito. Essa forma é útil para copiar ou salvar a estrutura da árvore.

void posOrdem(Arv A)

Realiza o percurso em pós-ordem (post-order): visita primeiro os filhos esquerdo e direito, e por último o nó atual. É geralmente usada para liberar memória da árvore.

void destroi(Arv *A)

Libera toda a memória ocupada pela árvore de forma recursiva. Após destruir cada nó, define o ponteiro como NULL para evitar referências inválidas.

void inserir(Item x, Arv *A)

Insere um novo elemento x na árvore binária de busca. Se a árvore estiver vazia, cria um novo nó; caso contrário, insere à esquerda se o valor for menor, ou à direita se for maior que o nó atual.

int busca(Item x, Arv A)

Realiza uma busca binária na árvore. Retorna 1 se o valor for encontrado e 0 caso contrário. A busca é feita comparando recursivamente o valor procurado com os nós da árvore.

Item removeMax(Arv *A)

Remove e retorna o maior elemento de uma árvore (ou subárvore), que sempre está no nó mais à direita. É usada como função auxiliar na remoção de nós com dois filhos.

void remover(Item x, Arv *A)

Remove um elemento x da árvore. Se o nó tiver um filho, ele é substituído por esse filho. Se tiver dois filhos, o valor do nó é substituído pelo maior valor da subárvore esquerda (obtido com removeMax). Se o valor não for encontrado, uma mensagem é exibida informando isso.

Considerações Finais:

Na prática, uma árvore binária é utilizada para organizar e acessar informações de forma rápida e eficiente. Ela é muito aplicada em situações em que é necessário armazenar e buscar dados de maneira estruturada.

Por exemplo, em um sistema de cadastro, cada registro (como o nome de um cliente ou o código de um produto) pode ser armazenado em um nó da árvore. Quando um novo dado é inserido, ele é colocado à esquerda ou à direita conforme seu valor, mantendo a ordem dos elementos. Assim, ao procurar um item específico, o sistema não precisa percorrer todos os registros — ele segue o caminho correto pela árvore, comparando valores até encontrar o desejado.

Esse tipo de estrutura também é usado em bancos de dados, compiladores, jogos e sistemas operacionais, pois permite reduzir o tempo de busca, inserção e remoção de informações, tornando os programas mais rápidos e organizados.

Código:

```
#include <stdio.h>
#include <stdlib.h>
#include "arvore.h"

Arv no(Arv A, Item x, Arv B){
    Arv novo = malloc(sizeof(struct arv));
    novo->esq = A;
    novo->item = x;
    novo->dir = B;
    return novo;
}

void emOrdem(Arv A){
    if(A != NULL){
        emOrdem(A->esq);
        printf("%d ", A->item);
        emOrdem(A->dir);
    }
}

void preOrdem(Arv A){
    if(A != NULL){
        printf("%d ", A->item);
        preOrdem(A->esq);
        preOrdem(A->dir);
    }
}

void posOrdem(Arv A){
    if(A != NULL){
        posOrdem(A->esq);
        posOrdem(A->dir);
        printf("%d ", A->item);
    }
}

void destroi(Arv *A){
    if(*A != NULL){
        destroi(&(*A)->esq);
        destroi(&(*A)->dir);
        free(*A);
        *A = NULL;
    }
}

void inserir(Item x, Arv *A){
    if(*A == NULL){
        *A = no(NULL, x, NULL);
    }else{
        if(x < (*A)->item){
            inserir(x, &(*A)->esq);
        }else{
            inserir(x, &(*A)->dir);
        }
    }
}

int busca(Item x, Arv A){
    if(A == NULL){
        return 0;
    }
    if(x == A->item){
        return 1;
    }
    if(x < A->item){
        return busca(x, A->esq);
    }else{
        return busca(x, A->dir);
    }
}

Item removeMax(Arv *A){
    if(*A == NULL){
        abort();
    }
    while( (*A)->dir != NULL){
        A = &(*A)->dir;
    }
    Arv aux = *A;
    Item x = aux->item;
    *A = aux->esq;
    free(aux);
    return x;
}

void remover(Item x, Arv *A){
    if(*A == NULL){
        printf("\nChave %d nao se encontra na arvore!\n", x);
        return;
    }
    if(x == (*A)->item){
        Arv aux = *A;
        if(aux->esq == NULL){
            *A = aux->dir;
        }else{
            if(aux->dir == NULL){
                *A = aux->esq;
            }else{
                aux->item = removeMax(&aux->esq);
            }
            if(aux != *A){
                free(aux);
            }
        }
    }else{
        if(x < (*A)->item){
            remover(x, &(*A)->esq);
        }else{
            remover(x, &(*A)->dir);
        }
    }
}
```